

AEP — ReUse+

Documento de Arquitetura, Requisitos e Planejamento

Instituição: UniCesumar — Faculdade de Engenharia de Software

Curso: Engenharia de Software

Integrantes: Guilherme Francisco Moreira Martinelli e Luiz Eduardo dos Santos Corrêa

Nome da equipe: A definir

Versão: 1.0 — Parte 1 / 1º Bimestre

Data de elaboração: 27 de agosto de 2026

Repositório: [Faculdade-de-engenharia-de-software-Unicesumar](#) — AEP

Nota metodológica. Como o grupo ainda não realizou entrevistas ou observações de campo, a descoberta descrita neste documento é um levantamento exploratório acadêmico, elaborado a partir de situações-problema plausíveis. Antes da entrega, o grupo poderá substituir as hipóteses por evidências obtidas com uma organização, projeto social ou comunidade local.

1. Descoberta e concepção

1.1 Contexto do problema

Pessoas, empresas e comunidades frequentemente possuem roupas, alimentos não perecíveis, materiais escolares, móveis e outros itens em boas condições que não utilizam mais. Ao mesmo tempo, instituições sociais e famílias em situação de vulnerabilidade precisam desses recursos, mas nem sempre conseguem localizar doadores, verificar a disponibilidade dos itens ou acompanhar o processo de recebimento.

Em um processo manual, as informações costumam ficar dispersas em mensagens, planilhas ou anotações. Essa fragmentação dificulta a atualização do status das doações, favorece a duplicidade de pedidos, reduz a rastreabilidade e aumenta o tempo gasto pelos voluntários na organização. O problema não é apenas registrar uma doação, mas conectar de maneira organizada **quem oferece, o que está disponível, quem necessita, qual instituição intermedeia e qual foi o destino do item.**

O ReUse+ será uma aplicação de apoio à gestão de doações e redistribuição de itens. Nesta primeira versão, seu objetivo não será realizar pagamentos, transporte ou validação jurídica de beneficiários. O sistema concentrará o cadastro, a consulta, a solicitação, a aprovação e o acompanhamento do ciclo de uma doação.

1.2 Partes interessadas

Parte interessada	Necessidade principal	Valor entregue pelo ReUse+
Doador	Oferecer itens e saber se foram encaminhados	Cadastro do item, consulta de status e histórico da doação
Instituição social	Organizar necessidades e aprovar solicitações	Catálogo centralizado, controle de solicitações e rastreabilidade
Voluntário	Apoiar triagem e encaminhamento	Registro das etapas de coleta, triagem e entrega
Beneficiário atendido	Receber o item adequado à necessidade	Solicitação intermediada pela instituição, sem exposição desnecessária de dados
Administrador	Manter cadastros e controlar permissões	Gestão de usuários, categorias e integridade dos registros

1.3 Matriz exploratória de descoberta

Certezas de projeto	Suposições a validar	Dúvidas para investigação futura
O sistema precisa registrar doações e seus status.	Instituições preferem uma aplicação simples em vez de múltiplas planilhas e mensagens.	Quais categorias de itens são mais frequentes na comunidade escolhida?
O histórico da destinação aumenta a rastreabilidade.	O processo normalmente possui etapas de cadastro, triagem, aprovação e entrega.	Quem será responsável por aprovar uma solicitação?
A aplicação precisa persistir os dados em um banco SQL real.	O acesso inicial poderá ocorrer por uma equipe pequena de voluntários.	A instituição necessita de relatórios impressos ou apenas consultas na aplicação?
O escopo deve ser implementável pelos dois integrantes no segundo bimestre.	Uma aplicação de console será suficiente para demonstrar arquitetura e CRUD em Engenharia de Software.	Será necessário incluir autenticação completa nesta versão acadêmica?

1.4 Jornada resumida do usuário

O doador acessa o sistema, informa seus dados e cadastra um item com categoria, descrição, condição e disponibilidade. A instituição consulta os itens, identifica uma necessidade e registra uma solicitação. Após a triagem, a solicitação pode ser aprovada ou recusada, e o item passa por estados controlados até ser marcado como entregue. Em cada etapa, o ReUse+ preserva o responsável e a data da alteração, permitindo consultar o histórico.

1.5 Regras de negócio

Código	Regra
RN01	Um item só pode ser associado a uma doação ativa por vez.
RN02	Uma doação não pode ser aprovada sem possuir doador, categoria, descrição e condição registrados.
RN03	Apenas uma instituição ou usuário com permissão administrativa pode aprovar ou recusar uma solicitação.
RN04	Uma solicitação aprovada reserva o item e impede que ele seja associado simultaneamente a outra solicitação aprovada.
RN05	O status deve seguir uma transição controlada: DISPONIVEL → EM_TRIAGEM → RESERVADO → ENTREGUE ; uma doação pode ser CANCELADA quando ainda não foi entregue.
RN06	A exclusão de registros que já possuem histórico deve ser lógica ou bloqueada, preservando a rastreabilidade.
RN07	O sistema deve evitar o armazenamento de dados sensíveis do beneficiário quando eles não forem necessários para a operação.

2. Alinhamento aos ODS

Os Objetivos de Desenvolvimento Sustentável são uma agenda global composta por 17 objetivos interconectados, voltados a enfrentar desafios sociais, econômicos e ambientais [1] [2]. O ReUse+ contribui principalmente para os seguintes objetivos:

ODS	Relação com o sistema
ODS 1 — Erradicação da pobreza	Facilita a organização de recursos destinados a pessoas em situação de vulnerabilidade, embora o software não pretenda medir ou eliminar a pobreza por si só.
ODS 10 — Redução das desigualdades	Ajuda instituições a direcionar itens disponíveis para necessidades registradas, reduzindo barreiras de acesso a recursos básicos.
ODS 12 — Consumo e produção responsáveis	Estimula o reaproveitamento e a extensão do ciclo de vida de itens que ainda possuem utilidade, reduzindo descarte prematuro.

A relação é de **apoio operacional**: o ReUse+ não substitui políticas públicas nem garante, sozinho, o cumprimento das metas dos ODS. Sua contribuição consiste em melhorar a organização, a visibilidade e a rastreabilidade de uma iniciativa local de redistribuição solidária.

3. Escopo funcional

3.1 Objetivo geral

Desenvolver uma aplicação orientada a objetos, com persistência em banco de dados SQL, capaz de organizar o cadastro e o acompanhamento de doações, solicitações e entregas realizadas por instituições sociais.

3.2 Requisitos funcionais

Código	Requisito
RF01	O sistema deve permitir o cadastro, a consulta, a alteração e a exclusão lógica de usuários, informando nome, e-mail, telefone, tipo de usuário e situação.
RF02	O sistema deve permitir o cadastro de itens para doação, informando descrição, categoria, condição de uso, quantidade e disponibilidade.
RF03	O sistema deve permitir consultar e filtrar doações por categoria, condição e status.
RF04	O sistema deve permitir que uma instituição registre uma solicitação de item, informando a instituição, o item desejado, a quantidade, a justificativa e o status da solicitação.
RF05	O sistema deve permitir que um responsável aprove, recuse ou cancele uma solicitação conforme as regras de negócio.
RF06	O sistema deve permitir atualizar o ciclo da doação entre os status disponível, em triagem, reservado, entregue e cancelado.
RF07	O sistema deve permitir registrar a entrega de uma doação, informando data, responsável, quantidade entregue e observações.
RF08	O sistema deve permitir consultar o histórico de alterações de status e os responsáveis por cada etapa da doação.
RF09	O sistema deve permitir emitir um resumo das doações por status, categoria e período.
RF10	O sistema deve impedir a aprovação de solicitações incompatíveis com a quantidade disponível ou com uma doação já reservada.

3.3 Requisitos não funcionais

Código	Requisito
RNF01	O sistema deve ser implementado com linguagem tipada e com aplicação explícita dos pilares da Orientação a Objetos.
RNF02	Os dados devem ser persistidos em um SGBD relacional compatível com SQL, MySQL 8.x.
RNF03	O sistema deve separar responsabilidades em camadas de apresentação, domínio e persistência.
RNF04	O código deve utilizar convenção <code>CamelCase</code> , nomes significativos, validações e tratamento de erros.
RNF05	O projeto deve conter instruções de instalação e execução no <code>README.md</code> .
RNF06	O repositório deve preservar o histórico de contribuições dos dois integrantes.
RNF07	O sistema não deve armazenar senha em texto puro nem dados pessoais desnecessários para a operação.

3.4 Fora do escopo da primeira versão

A primeira versão não contemplará pagamentos, aplicativo móvel nativo, geolocalização em tempo real, integração com transportadoras, cálculo de impacto ambiental certificado, autenticação multifator ou publicação automática em lojas de aplicativos. Esses itens poderão ser registrados como oportunidades futuras, mas não farão parte do contrato funcional da Parte 1.

4. Planejamento ágil do segundo bimestre

Como as datas oficiais não foram informadas, o cronograma abaixo é **simulado e antecipado**, devendo ser ajustado ao calendário divulgado pelo professor. Ele organiza o desenvolvimento em quatro sprints de duas semanas.

Sprint / período sugerido	Épico	User story / atividade	Responsável
Sprint 1 — 31/08 a 11/09/2026	Fundação e usuários	Como administrador, quero cadastrar e consultar usuários para controlar quem participa do processo de doação. Criar projeto, conexão SQL, entidades e repositórios iniciais.	Guilherme
Sprint 2 — 14/09 a 25/09/2026	Catálogo de doações	Como doador, quero cadastrar e consultar itens para disponibilizá-los a instituições. Implementar RF02, RF03 e CRUD principal.	Luiz
Sprint 3 — 28/09 a 09/10/2026	Solicitações e regras	Como instituição, quero solicitar itens e acompanhar a aprovação para atender necessidades registradas. Implementar RF04, RF05 e validações RN03, RN04 e RN10.	Guilherme e Luiz
Sprint 4 — 12/10 a 23/10/2026	Entrega e relatórios	Como responsável, quero registrar a entrega e consultar o histórico para comprovar a destinação. Implementar RF06–RF09, testes, README e integração final.	Guilherme e Luiz
Reserva — 26/10 a 30/10/2026	Estabilização	Corrigir defeitos, validar o script SQL, revisar documentação, testar execução em ambiente limpo e preparar a apresentação.	Guilherme e Luiz

O cronograma distribui responsabilidades sem impedir a colaboração. Cada integrante deverá realizar commits identificáveis no GitHub, revisar o código do colega e registrar no README eventuais alterações justificadas de escopo.

5. Justificativa técnica e arquitetural

5.1 Linguagem e paradigma

Propõe-se utilizar **Java** no segundo bimestre, com uma aplicação inicialmente executada via terminal. Java oferece tipagem estática, ampla documentação e suporte direto aos conceitos

exigidos na AEP. A linguagem permite demonstrar uma classe abstrata ou interface, herança, encapsulamento, composição e polimorfismo por sobrescrita de métodos com `@Override`.

O domínio foi modelado com uma superclasse abstrata `Usuario`, da qual podem derivar `Doador`, `Voluntario`, `Instituicao` e `Administrador`. Cada subtipo poderá sobrescrever operações como `obterPermissoes()` ou `descreverAtuacao()`, demonstrando polimorfismo de forma justificável. A classe `Doacao` será composta por seus registros de histórico, caracterizando uma relação de composição 1:N.

5.2 Banco de dados

O SGBD adotado será o **MySQL 8.x**, utilizando SQL para definição e manipulação dos dados. A escolha se justifica pela ampla adoção acadêmica e profissional, pela integridade referencial com o mecanismo InnoDB, pelo suporte a restrições `CHECK`, transações e pela compatibilidade com uma aplicação Java via JDBC. O banco será necessário para que o CRUD não seja simulado em listas temporárias ou arquivos de texto, conforme exigido nas diretrizes.

As tabelas serão normalizadas para separar usuários, categorias, itens, doações, solicitações, entregas e histórico. Chaves estrangeiras preservarão os relacionamentos, enquanto índices em colunas de busca — como status, categoria e datas — poderão melhorar as consultas. O script inicial ficará em `/database/001_schema.sql` e será escrito para MySQL 8.x com tabelas InnoDB, `AUTO_INCREMENT`, chaves estrangeiras, índices e restrições compatíveis com o SGBD.

5.3 Arquitetura da aplicação

A solução adotará um **monólito modular em camadas**, mais adequado ao tamanho da equipe e ao prazo acadêmico do que uma arquitetura de microsserviços. A divisão prevista será:

Camada	Responsabilidade
Apresentação	Menus de terminal, leitura de entradas e mensagens ao usuário.
Aplicação/Serviço	Orquestra casos de uso, valida permissões e controla transações.
Domínio	Entidades, interfaces, herança, regras de negócio e estados da doação.
Persistência	Conexão JDBC, repositórios, consultas parametrizadas e mapeamento dos dados.
Banco	Tabelas, chaves, restrições, índices e script de criação.

Essa escolha mantém o projeto executável e compreensível, reduz o acoplamento entre menu e banco e torna possível testar as regras de negócio de forma isolada. A arquitetura também deixa aberta uma evolução futura para uma API ou interface web sem reescrever o núcleo do domínio.

5.4 Segurança, confiabilidade e operação

Mesmo em uma aplicação acadêmica de console, o projeto deverá adotar consultas parametrizadas para reduzir riscos de injeção SQL, validação de campos obrigatórios, controle de permissões por tipo de usuário e tratamento explícito de erros de conexão. Senhas, caso sejam implementadas, deverão ser armazenadas somente como hash; dados sensíveis de beneficiários devem ser minimizados.

O `README.md` deverá explicar a versão do Java, a criação do banco MySQL 8.x, a configuração da conexão, a execução do script e o comando para iniciar a aplicação. O sistema deverá falhar de maneira compreensível quando o banco estiver indisponível, sem mascarar o erro como se a operação tivesse sido realizada.

6. Estrutura do repositório

O repositório deverá conter a seguinte estrutura:

```
AEP-2 Semestre - 2º Ano/
├── README.md
├── src/
│   └── (código Java da aplicação no 2º bimestre)
├── docs/
│   ├── PARTE_1_AEP_REUSE.md
│   ├── diagramas/
│   │   ├── diagrama-classes.mmd
│   │   └── diagrama-der.mmd
│   └── pdf/
│       └── AEP_ReUse_Parte_1.pdf
└── database/
    └── 001_schema.sql
```

O link público de referência é: [GitHub — AEP ReUse+](#).

7. Critérios de aceite da Parte 1

Critério	Evidência prevista
Escopo e ODS	Problema descrito, ODS relacionados e dez requisitos funcionais rastreáveis.
Cronograma	Tabela de sprints, datas sugeridas, atividades e responsáveis no PDF e no README.
Justificativa técnica	Linguagem Java, MySQL/SQL, arquitetura em camadas e segurança justificadas pelo escopo.
GitHub	Repositório público com <code>/src</code> , <code>/docs</code> , <code>/database</code> e README.
Diagrama de Classes	Herança, polimorfismo e composição 1:N identificados e compatíveis com os requisitos.
DER	Entidades, chaves primárias, estrangeiras e cardinalidades coerentes com o domínio.

8. Rastreabilidade inicial

Requisito	Entidades principais	Sprint
RF01	Usuario, Doador, Voluntario, Instituicao, Administrador	1
RF02–RF03	Categoria, Item, Doacao	2
RF04–RF05 e RF10	Solicitacao, Instituicao, Doacao	3
RF06–RF08	Doacao, Entrega, HistoricoStatus	4
RF09	Doacao, Categoria, HistoricoStatus	4

Referências

[1]: <https://brasil.un.org/pt-br/sdgs> — Nações Unidas no Brasil. “Objetivos de Desenvolvimento Sustentável”. Acesso em 27 ago. 2026.

[2]: <https://sdgs.un.org/goals> — United Nations. “The 17 Goals”. Acesso em 27 ago. 2026.